



Erstellung und Standardisierung eines Roboterablaufes zum Depalettieren von Verpackungseinheiten

Projektarbeit T2000

über die Praxisphase des zweiten Studienjahrs

an der Fakultät für Technik
im Studiengang Informationstechnik

an der DHBW Ravensburg
Campus Friedrichshafen

von
Eric Erath

9. September 2024

Bearbeitungszeitraum:	01. 07. 2024 - 02. 08. 2024
Matrikelnummer, Kurs:	1910897, TIT22
Dualer Partner:	fpt Systems GmbH
Betreuer des Dualen Partners:	Matthias Rundel, Simon Würzer

Sperrvermerk

gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017.

„Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung vom Dualen Partner vorliegt.“

Hergensweiler, 9. September 2024

Ort, Datum

SW-Operations, 

Abteilung, Unterschrift

Selbstständigkeitserklärung

gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017.

Ich versichere hiermit, dass ich meine Projektarbeit mit dem Thema:

Erstellung und Standardisierung eines Roboterablaufes zum Depalettieren von Verpackungseinheiten

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Hergensweiler, 9. September 2024

Ort, Datum

SW-Operations, 

Abteilung, Unterschrift

Zusammenfassung

Die Firma „FPT Robotik GmbH Co. KG“ entwirft, programmiert und verkauft industrielle Roboteranlagen. In einer neuen Anlage für den Kunden „Julius Blum GmbH“ befindet sich ein Roboterarm, der sowohl Verpackungseinheiten depalettieren, als auch Deckel auf Stülpkartons auffädeln muss.

In diesem Projekt soll der Depalettierablauf des Roboters erstellt und getestet werden. Auf Wunsch der „Julius Blum GmbH“ soll der das Programm mithilfe der drei Programmiersprachen „Kuka Robot Language“, „Graphic Motion Language“ und „Structured Text“ erstellt werden. Das Vorgehen zum Erstellen des Programms soll dokumentiert werden, um als Leitfaden für andere Anlagenprogrammierer zu dienen.

Aufgrund von Zeitmangel konnte der Depalettierablauf nicht fertiggestellt werden. Das allgemeine Vorgehen wird dennoch in dieser Arbeit und den erstellten Leitfaden dokumentiert.

Schlüsselwörter

„Kuka Robot Language“ (KRL), Sechachsroboter, „Graphic Motion Language“ (GML), Depalettierablauf, „Structured Text“ (ST), „Speicherprogrammierbare Steuerung“ (SPS), Roboterprogrammierung

Abstract

The company „FPT Robotik GmbH Co. KG“ designs, programs and sells industrial robot-systems. In a new system for the customer „Julius Blum GmbH“ there is a robot arm that has to depalletize packaging units as well as thread lids onto cardboard boxes.

In this project, the robot's depalletizing sequence is to be created and tested. At the request of the „Julius Blum GmbH“, the program will be created using the three programming languages „Kuka Robot Language“, „Graphic Motion Language“ and „Structured Text“. The necessary steps for creating the program should be documented in order to serve as a guide for other system programmers.

Due to a lack of time, the depalletising process could not be completed. Nevertheless, the general procedure is documented in this work and the created guidelines.

Keywords

„Kuka Robot Language“ (KRL), six-axis-robot, „Graphic Motion Language“ (GML), depalletizing sequence, „Structured Text“ (ST), „programmable logic controller“(PLC), robot-programming

Vorwort

In dieser Projektarbeit wird bewusst auf die Verwendung von geschlechterspezifischen Formulierungen verzichtet und das generische Maskulinum verwendet. Dies soll keinesfalls als gewollte Ausgrenzung von Gruppierungen gedeutet werden, sondern zur Lesbarkeit und Vermeidung von umständlichen Formulierungen beitragen. Weibliche und anderweitige Geschlechteridentitäten sind dabei ausdrücklich mitgemeint. Der Fokus dieser Arbeit liegt gänzlich auf der neutralen und wissenschaftlichen Behandlung des Themas, nicht auf der korrekten Einbindung aller Geschlechter.

Inhaltsverzeichnis

Sperrvermerk	II	6 Zusammenfassung zum Leitfaden	16
Selbstständigkeitserklärung	III	6.1 Inhalt	16
Zusammenfassung	IV	6.2 Anhänge	17
Abstract	IV	6.3 Verteilung	18
Vorwort	V	7 Reflexion	19
Abkürzungsverzeichnis	VII	7.1 Zusammenfassung	19
Abbildungsverzeichnis	VIII	7.2 Nötige Ergänzungen	19
1 Grundlagen	1	8 Fazit	21
1.1 Funktionsweise von GML . . .	1	Literatur	22
1.2 Jobs und Types	2	A Anhang	A
2 Problemstellung	3		
3 Vorbereitungen	4		
3.1 Erstellung des Greifer-Codes .	4		
3.2 Verknüpfung der Variablen mit Sensorik/Aktorik	5		
4 Programmierung in KRL	6		
4.1 Jobs	6		
4.2 Basiskoordinatensysteme . . .	7		
4.2.1 Erstellung von Basen .	8		
4.2.2 Vorteile	9		
4.3 Toolkoordinatensysteme . . .	9		
4.4 Lastdaten	10		
4.5 Höhengsuchfahrt	11		
5 Programmierung in GML	12		
5.1 Variablen	12		
5.2 Auswahl der Jobs	13		
5.3 Durchführung der Jobs	14		

Abkürzungsverzeichnis

ADS	„Automation Device Specification“	1
Beckhoff	„Beckhoff Automation GmbH Co. KG“	2
Blum	„Julius Blum GmbH“	1
BPMN	„Business Process Modeling Notation“	2
CAD	„Computer-aided Design“	10
fpt	„FPT Systems GmbH“	1
GFT	„Gebideförderertechnik“	6
GML	„Graphic Motion Language“	IV
KRL	„Kuka Robot Language“	IV
Kuka	„KUKA AG“	2
MD	„Markdown“	16
OPCUA	„Open Platform Communication Unified Architecture“	1
Siemens	„Siemens AG“	2
SPS	„Speicherprogrammierbare Steuerung“	IV
ST	„Structured Text“	IV
Stiwa	„STIWA Holding GmbH“	1
TCP	„Tool Center Point“	9
Word	„Microsoft Word“	16

Abbildungsverzeichnis

1.1	Anlagenübersicht der FA3278	1
3.1	Zeichnung des Greifaufsatzes	4
3.2	Anschlüsse an der Greifer	5
4.1	Beispiel für Basis- und Weltkoordinaten	7
4.2	Nummerierung von Oktanten	8
4.3	Beispiel für Flansch- und Werkzeugkoordinaten	9
4.4	ECO-Packs auf Paletten	11
5.1	Für GML eingerichtete globale Variablen	12
5.2	Code der Abfrage „Job1?“, aus Abbildung 5.4	13
5.3	GML-Code zur Auswahl eines Jobs	14
5.4	Hauptablauf des Roboters in GML	15

1 Grundlagen

Die Firma „FPT Robotik GmbH & Co. KG“ spezialisiert sich auf das Entwickeln von Roboteranlagen für internationale Kunden. Die Programmierung dieser Anlagen wird von der Tochterfirma „FPT Systems GmbH“ (fpt) übernommen. Hierbei übernimmt eine SPS die Steuerung der Peripherie der Anlage und kommuniziert mit der Robotersteuerung. Die eigens geschriebene Software FlexOP bietet eine grafische Oberfläche zur Konfiguration, Überwachung und Programmierung. In älteren Anlagen findet die Kommunikation zwischen den beiden Programmen hauptsächlich über den Standard „Open Platform Communication Unified Architecture“ (OPCUA) statt. Neuere Anlagen verwenden stattdessen vermehrt das Protokoll „Automation Device Specification“ (ADS).

Diese Arbeit behandelt den Depalettierroboter im Modul M54 der Anlage FA3278 für Blum.

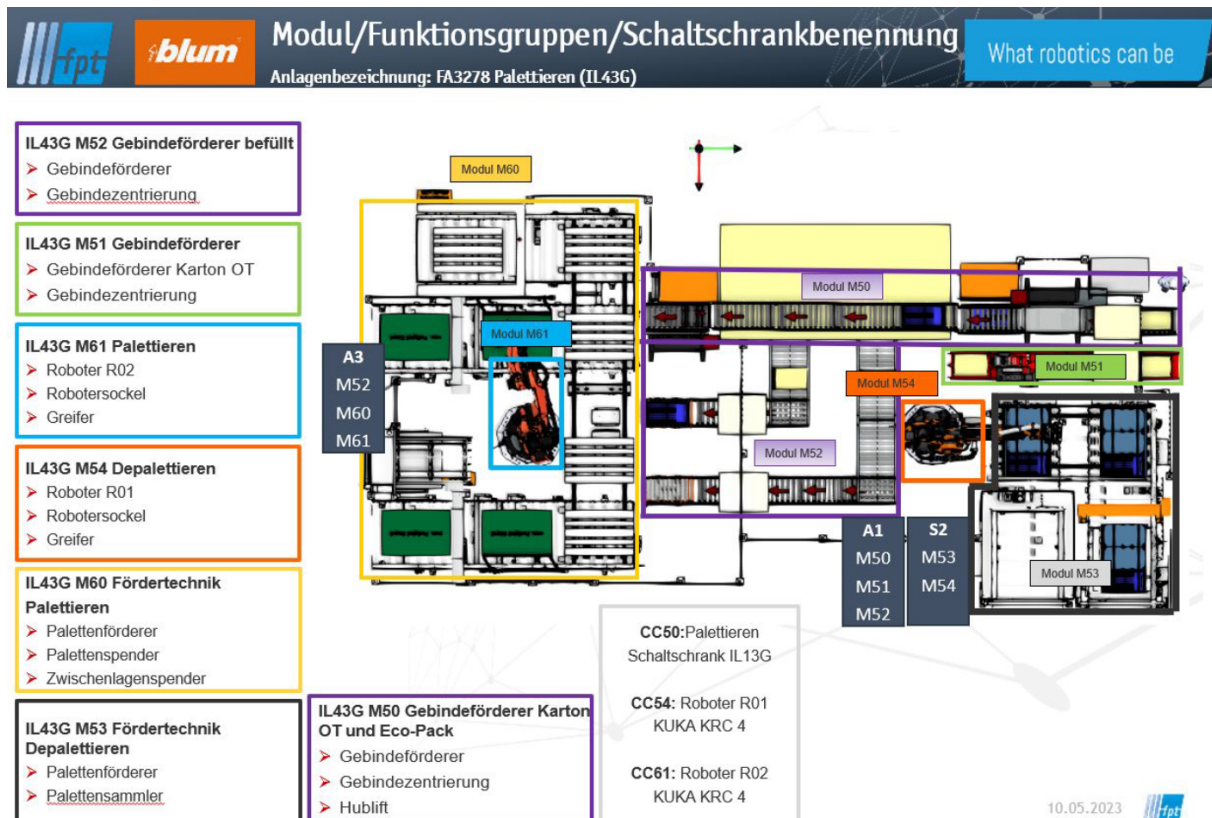


Abb. 1.1: Anlagenübersicht der FA3278 (FPT Robotik GmbH Co. KG 2023, S. 2)

1.1 Funktionsweise von GML

Auf Wunsch der Kundenfirma „Julius Blum GmbH“ (Blum) wird der Automatikablauf der Anlage in der GML erstellt. GML ist eine von der „STIWA Holding GmbH“ (Stiwa) entwickelte Programmiersprache, zur Ablaufprogrammierung von SPS-gesteuerten Roboteranlagen. Die Spra-

che ist Kompatibel mit SPSen der beiden dominierenden Herstellern, „Siemens AG“ (Siemens) und „Beckhoff Automation GmbH Co. KG“ (Beckhoff), und versucht so die Produktionsprogrammierung zu standardisieren. GML ist zudem in Aussehen und Funktionsweise ähnlich zur „Business Process Modeling Notation“ (BPMN), einer standardisierten Modellierungssprache für Geschäftsprozesse und daher ähnlich leicht zu lernen und verwenden. Stiwa verspricht durch Verwendung von GML unter anderem „25 % Reduktion der SPS-Konstruktions-Aufwände [und] 20 % Reduktion der Inbetriebnahme-Aufwände“. (STIWA Holding GmbH 2024) Blum erhofft sich so viele Wartungen oder sonstige Änderungen an der Anlage, ohne die Hilfe von fpt durchführen zu können.

Um Ausgänge, wie beispielsweise das Einschalten eines Saugers oder Förderbandes, zu schalten verfügt GML über Codeelemente, sogenannte Shapes, die SPS-Code ausführen können. So werden die Variablen in der SPS direkt beschrieben. Auf diese Weise können auch Messwerte von Sensoren in GML abgefragt werden.

Zur Bewegung des Roboterarms reicht dieses Feature nicht aus, da der Roboter nicht von der SPS, sondern von einer spezialisierten Robotersteuerung von der „KUKA AG“ (Kuka) gesteuert wird. Um den Arm über GML zu bewegen, werden in einem Shape die Typnummer und die Jobnummer angegeben. Über diese Nummern wird der Robotersteuerung mitgeteilt, welche Verfahrerroutine angewählt werden soll. Zusammen mit den beiden Nummern werden an die Steuerung noch andere Parameter übergeben, wie Offsets zum nächsten Gegenstand, der von einer Palette genommen werden soll oder die korrekte Anfahrtrichtung und Orientierung zum Depalettieren.

1.2 Jobs und Types

Jobs werden in der KRL programmiert. Die Programmiersprache wurde von Kuka zur Programmierung ihrer Roboterarme entwickelt und bietet viele nützliche Features, wie das Transformieren von Koordinatensystemen nativ an.

Jobs übernehmen das Verfahren des Roboterarms, indem sie aus verschiedenen vorgegebenen Startpositionen in eine festgelegte Endposition fahren. Hierbei stellt die Grundstellungsfahrt, als Job eins in Typ eins, eine Ausnahme dar. Die Grundstellungsfahrt, oder auch das Safe-Home-Programm, kann aus jeder Position aufgerufen werden und bringt den Roboterarm in die konfigurierbare Grundstellung. Um nach einem Abbruch mit folgender Grundstellungsfahrt den Ablauf korrekt fortsetzen zu können ist es meist gegeben, dass jeder Job mindestens aus der Grundstellung anwählbar ist. Zur besseren Übersichtlichkeit werden zusammengehörige Jobs in Typen sortiert. Beispielsweise befinden sich die Jobs zum Depalettieren, bei der für diese Arbeit relevanten Anlage, in Typ vier. Jobs werden zudem in Steps unterteilt. So kann über GML beispielsweise in Step drei die Klemmen des Greifers schließen, um einen Karton aufzuheben.

2 Problemstellung

Im Laufe des Projektes soll ein Roboterablauf zum Depalettieren von Verpackungseinheiten im Kontext einer sich in der Entwicklung befindenden Roboteranlage für den Kunden Blum erstellt werden.

Der Depalettierablauf soll, auf Wunsch von Blum, in den Sprachen KRL und GML implementiert werden. KRL soll die Bewegung des Roboters übernehmen, während über GML die logische Steuerung der Anlage geschehen soll.

Neben der Planung und praktischen Umsetzung des Ablaufes soll zudem ein Leitfaden zur standardisierten Erstellung dieser Ablaufart erstellt werden. Dieser Leitfaden soll Programmierern der fpt künftig sowohl eine Hilfestellung bei der Umsetzung ähnlicher Abläufe bieten, sowie die Programmierung dieser Ablaufart vereinheitlichen. Die programmiererübergreifende Verständlichkeit der entstehenden Programme soll somit verbessert werden.

3 Vorbereitungen

Zu Beginn der Projektarbeit war die Roboteranlage nicht bereit über GML beziehungsweise KRL programmiert zu werden. Das SPS-Programm, in dem der Hauptablauf in GML integriert wird und das als Schnittstelle zum Programm in KRL dient musste erst noch erstellt werden.

Als Hilfestellung wird hierfür der Code eines anderen Projektes für Blum kopiert und nach Bedarf angepasst.

3.1 Erstellung des Greifer-Codes

Da das kopierte Projekt für einen Roboterarm mit anderem Greiferaufsatz erstellt wurde, muss unter anderem der zugehörige Code ersetzt werden.

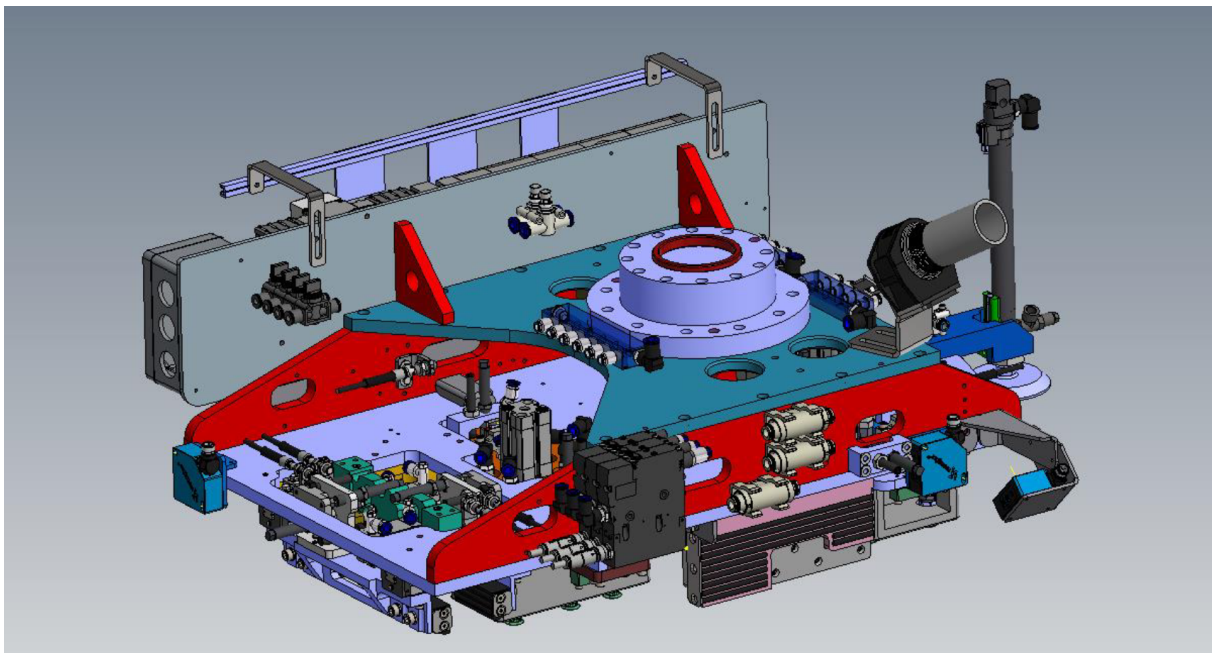


Abb. 3.1: Zeichnung des Greiferaufsatzes (FPT Robotik GmbH Co. KG 2024, S. 1)

Der verwendete Greifer verfügt über drei ausfahrbare Saugkreise, zwei Klemmen und einen Höhenmesszylinder. Zum Depalettieren werden nur die beiden Klemmen und der Höhentaster benötigt. Die fpt besitzt zur Ansteuerung der pneumatischen Zylinder mit denen die Klemmen geöffnet und geschlossen werden, beziehungsweise der Messzylinder aus- und eingefahren wird, vorgefertigte Funktionsbausteine, denen nur Steuerungsparameter von FlexOP und die Istposition der Zylinder übergeben, sowie die Ausgänge der Bausteine mit den korrekten Hardwareausgängen der SPS verknüpft werden muss.

Zudem müssen noch Verweise auf den Greiferbaustein und die einzelnen Zylinder in der, von FlexOP verwendeten, Projektdatenbank eingefügt werden. Anschließend kann FlexOP die Werte der Sensoren anzeigen und die Zylinder im Handbetrieb manuell steuern. So kann überprüft

werden, dass alle Sensoren und Aktoren richtig an die Steuerung angeschlossen, und auch nicht defekt, sind. Anschließend kann im Automatikbetrieb über das SPS-Programm und somit auch in GML die Hardware verwendet werden.

3.2 Verknüpfung der Variablen mit Sensorik/Aktorik

Wie in 3.1 bereits erwähnt müssen einige Variablen im SPS-Code mit Messwerten von Sensoren beziehungsweise Signale an Pneumatikventile oder ähnliches verbunden werden, sodass die SPS in der Lage ist den Istzustand des Greifers zu überprüfen und die Klemmen, Zylinder und Saugerkreise zu steuern.

Bei fpt-Projekten gibt es standardmäßig eine Liste globaler Variablen in der SPS, in der die entsprechenden Variablen über den Zusatz „AT %I*“ als Hardwareeingang beziehungsweise mit „AT %Q*“ als Ausgang markiert werden. Die Verknüpfung geschieht dann über den sogenannten Sys-Manager, ein separates SPS-Projekt, das die verschiedenen Projekte der Roboterarme und Fördertechniken einer Anlage auf derselben SPS laufen lässt, und zudem die Hardwareanschlüsse und deren Verknüpfung verwaltet.

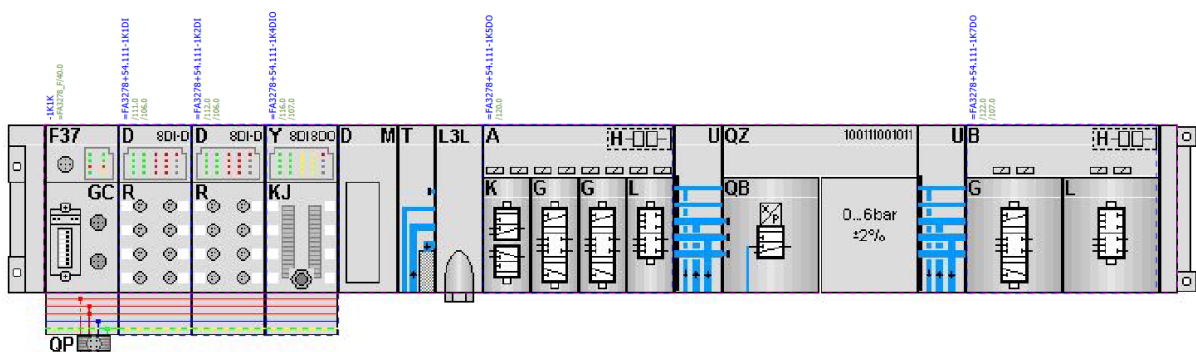


Abb. 3.2: Anschlüsse an den Greifer (FPT Robotik GmbH Co. KG 2023, S. 266)

Die Abbildung zeigt eine Zeichnung der Anschlüsse am Greifer aus dem Planungsdokument der fpt zur Elektronik und Pneumatik. Die einzelnen Geräte, deren IDs in Blau über den Elementen stehen, kommunizieren über den EtherCAT-Bus mit der SPS und verarbeiten die digitalen Signale so, dass beispielsweise eine LED leuchtet oder das Ventil eines Zylinders geöffnet wird. Auch die Sensoren werden hier angeschlossen und übergeben ihre Messwerte in digitaler Form über den Bus an die SPS.

Der Sys-Manager kann mithilfe von Software der Firma Beckhoff den Bus scannen, und so die verfügbaren Anschlüsse identifizieren. Zuletzt muss im Systemmanager nur bei den einzelnen Anschlüssen der Name der entsprechende Variable eingetragen werden.

4 Programmierung in KRL

Wie bereits erwähnt wird in der Kuka-Steuerung KRL zum Bewegen des Roboters verwendet, und die Steuerung erhält von GML die nötigen IDs für Job und Typ. Mit SWITCH-Statements wird aus den übergebenen Nummern für Job und Typ, der korrekte Ablauf ausgewählt. Im Fall, dass der entsprechende Job oder Typ nicht existiert wird der Vorgang abgebrochen und der Anlagenprogrammierer erhält eine Fehlermeldung.

4.1 Jobs

In den einzelnen Jobs wird zunächst überprüft, ob der Roboter sich in einer unterstützten Ausgangsstellung befindet. Im Negativfall kann nicht garantiert werden, dass der Roboter kollisionsfrei ans Ziel kommt und der Vorgang wird mit einer Fehlermeldung an den Programmierenden abgebrochen.

Befindet sich der Arm in einer validen Position ist der nächste Schritt den Roboter „frei zu fahren“, das bedeutet zu einem Punkt, der im Normalfall noch nicht der Zielposition entspricht, aber auf einer sinnvollen Route zwischen allen validen Startpunkten und dem Zielpunkt liegt. Das erreicht, dass die letzte Fahrt zum Zielpunkt nicht für jeden Startpunkt einzeln zu programmieren ist, sondern die gleiche Zielfahrt für alle Startpositionen nach dem Freifahren erfolgt.

Die Aufteilung in Frei- und Zielfahrt ist jedoch nicht immer sinnvoll, denn dadurch entstehen meist Umwege in der Fahrbahn, die die Zykluszeit des Roboters beeinträchtigen. In Extremfällen muss also für jeden Startpunkt eine eigene, möglichst optimale Strecke zum Zielpunkt implementiert werden.

Nicht selten müssen zum Verfahren Positionen zur Laufzeit berechnet werden. Platzhalter für diese Punkte werden im Gegensatz zu den konfigurierten Positionen, auch Teachpositions genannt, nicht in der Datei „ApplicationTeach.dat“, sondern in der „ApplicationRuntime.dat“ gespeichert. Dies ist nötig, da KRL keine Speicherallozierung zur Laufzeit unterstützt. Die eigentliche Berechnung geschieht standardmäßig in einer anderen Codedatei als der des zugehörigen Typs, beispielsweise befindet sich der Code zum Depalettieren in der Datei „u_Type4_Depal.src“ und die zugehörigen Berechnungen in „u_Type4_Calculation.src“.

Während manchen Jobs müssen über GML Klemmen oder ähnliches angesteuert werden. Wie bereits erwähnt werden die Jobs dazu in Steps unterteilt. Die Nummer des aktuellen Steps kann im Hauptablauf abgefragt werden. Alternativ kann das Shape zum Starten eines Jobs konfiguriert werden, den Hauptablauf zu pausieren, bis ein bestimmter Step des Jobs erreicht ist. Im Fall dieses Projektes sind Ansteuerungen der Klemmen nur zwischen verschiedenen Jobs nötig.

Der erste Job beim Depalettieren bewegt den Arm aus der Grundstellung, Warteposition oder der Ablageposition auf der „Gebidefördertechnik“ (GFT) zur Palette und positioniert den

Greifer so, dass nach Ende des Jobs durch Schließen der Klemme das nächste Teil auf der Palette gegriffen wird.

Job zwei Führt den Roboter von der Grundstellung oder der letzten Aufnahme position auf der Palette zur Ablageposition auf der GFT. Nach dem Ende dieses Jobs muss die Klemme geöffnet werden und der Greifer über einen anderen Job aus dem Weg gefahren werden. Ansonsten blockieren Teile des Greifers den Weg auf der GFT.

Nach dem Ablegen kann der Roboter direkt wieder zur Palette gefahren werden, um die nächste Verpackungseinheit zu Depalettieren. Wenn keine Teile mehr auf der Palette sind, kann stattdessen über Job drei eine Warteposition über der Palette angefahren werden. Die Warteposition ist auch aus der Grundstellung mithilfe vom dritten Job erreichbar.

4.2 Basiskoordinatensysteme

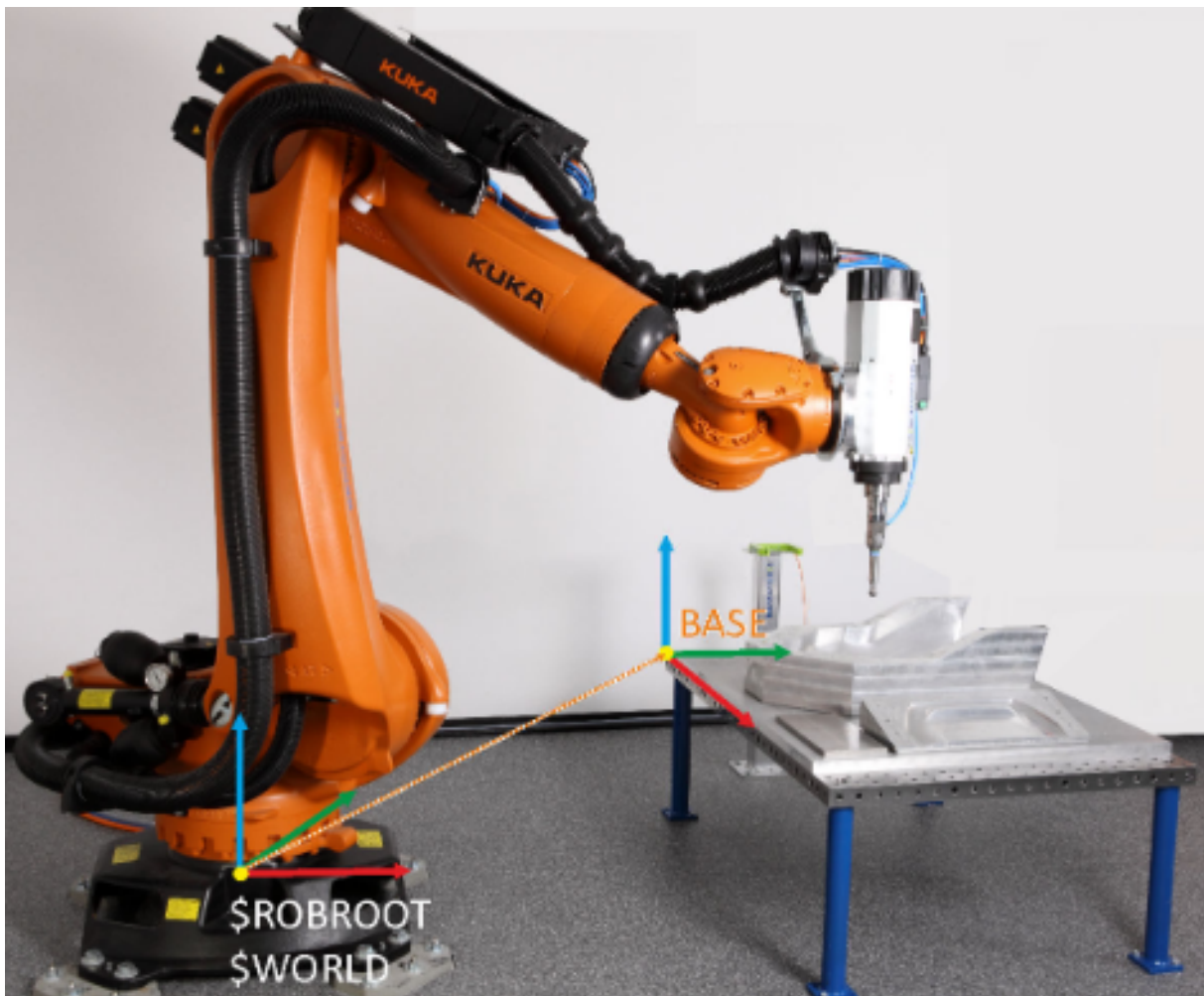


Abb. 4.1: Beispiel für Basis- und Weltkoordinaten (KUKA Deutschland GmbH 2021, S. 148)
(x-Achse / y-Achse / z-Achse)

Kukaroboter berechnen Positionen immer in einem kartesischen Rechtshandkoordinatensystem,

mit einem Millimeter pro Längeneinheit. Das Grundkoordinatensystem wird in KRL „\$WORLD“ genannt. Relativ zu diesem System können andere Koordinatensysteme, sogenannte Bases, erstellt werden.

4.2.1 Erstellung von Basen

Bases lassen sich, unter anderem, durch die Vermessung von drei Punkten von der Robotersteuerung berechnen. Der erste Punkt wird zum Ursprung des neuen Koordinatensystems. Der Vektor zwischen dem ersten und zweiten Messpunkt bestimmt die Richtung x-Achse der Basis, die Länge des Vektors ist egal, da auch in der neuen Basis eine Längeneinheit von eins, einem Millimeter realer Bewegung entspricht. Die dritte Position muss sich nur noch in der xy-Ebene befinden, mit positivem y-Offset. Daraus berechnet die Robotersteuerung zunächst die y-Achse, die immer senkrecht auf der x-Achse steht. In der xy-Ebene gibt es jedoch zwei Möglichkeiten für eine orthogonale y-Achse, daher muss der dritte Teachpunkt in positiver y-Richtung verschoben sein, so kann die richtige Richtung gewählt werden. Zuletzt wird die z-Achse berechnet, die im rechten Winkel auf den beiden anderen Achsen steht, auch hier gibt es zwei mögliche Richtungen und es wird die gewählt, bei der das entstehende Koordinatensystem der Rechten-Hand-Regel entspricht. (KUKA Deutschland GmbH 2021, S. 153ff.)

Zum Depalettieren wird bei fpt standardmäßig eine Basis für die zu bearbeitende Palette erstellt. Hierbei liegt eine der oberen Ecken der Palette im Ursprung des Koordinatensystems und die Palette erstreckt sich in die positive x- und y-Richtung mit der Oberfläche in der Koordinatenebene $z=0$. Die Palette befindet sich also im minus-fünften Oktanten und die zu depalettierenden Gegenstände befinden sich im dreizehnten Oktanten. (Sh. Abb. 4.2).

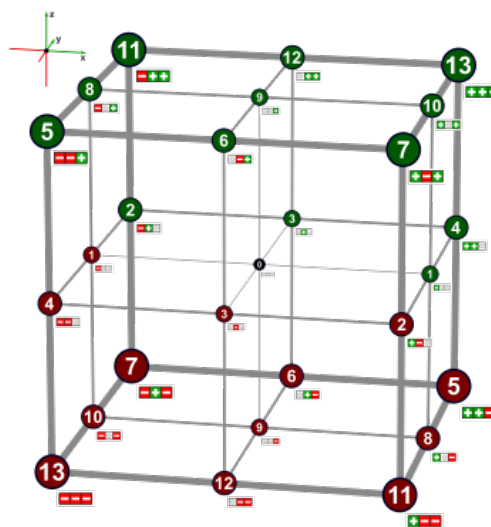


Abb. 4.2: Nummerierung von Oktanten
([de.wikipedia.org/wiki/Oktant_\(Geometrie\)](https://de.wikipedia.org/wiki/Oktant_(Geometrie)))

4.2.2 Vorteile

Im Fall, dass der Depalettierplatz relativ zum Roboter verschoben wird, reicht es aus die Basis anzupassen, da alle Teachpositionen die sich relativ zur Palette befinden sollen, in der Palettenbasis konfiguriert wurden, und somit bei einer Verschiebung der Base mit verschoben werden. (KUKA Deutschland GmbH 2021, S. 149f.)

Zudem werden so einige Berechnungen für palettenbezogene Punkte erleichtert, da Offsets und Rotationen, die beim Arbeiten in Weltkoordinaten nötig wären, wegfallen. Beispielsweise befindet sich die Mitte der Plattenoberfläche bei den Koordinaten

$$\left(\frac{Palettenlaenge_1}{2} \mid \frac{Palettenlaenge_2}{2} \mid 0 \right) \quad (1)$$

und die Oberkante der n -ten Schicht auf der Palette liegt bei

$$z = n * Lagenhoehe \quad (2)$$

Die nötigen Offsets zum Greifen des nächsten Teils auf der Palette, werden jedoch nicht in der Robotersteuerung, sondern in der SPS, mithilfe von firmeninternen Bibliotheken, berechnet und an die Robotersteuerung übergeben, die dann die Offsets beim Verfahren auf den Palettenursprung addiert.

4.3 Toolkoordinatensysteme

Ähnlich zu Basen, die das Koordinatensystem relativ zu den Weltkoordinaten bestimmen, kann mithilfe von Toolkoordinatensystemen der „Tool Center Point“ (TCP) relativ zum Flansch des Roboterarms rotiert und verschoben werden.

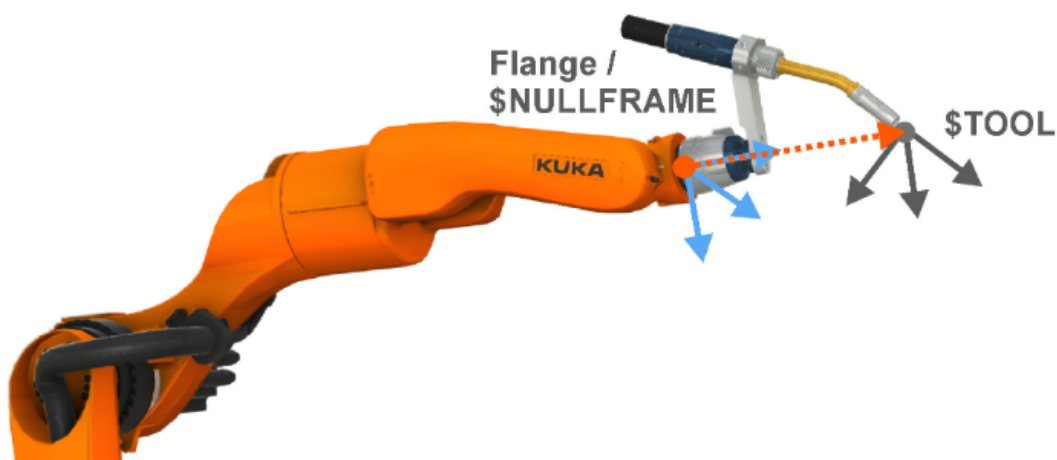


Abb. 4.3: Beispiel für Flansch- und Werkzeugkoordinaten (KUKA Deutschland GmbH 2021, S. 60)

TCP ist ein anderer Name für den Ursprung des Werkzeugkoordinatensystems. Der TCP ist zudem der Punkt am Roboter, der am Ende einer Fahrbewegung auf den Zielkoordinaten und auch in der Zielorientierung befindet. Das bedeutet, dass eine Verschiebung und Rotation des TCP relativ zum Flansch auch als eine invertierte Verschiebung beziehungsweise Rotation des Flanschs relativ zum Ziel der Roboterbewegung ist.

Diese Verschiebung hat den Vorteil, dass beim Berechnen von Zielpositionen der Greifer als Bestandteil des Roboterarmes betrachtet werden kann und somit Offset und Rotation des Tools vom Programmierer nicht beachtet werden müssen. Zudem kann, wie bei den Bases, eine Änderung am Greiferaufsatz durch Anpassen der Werkzeugkoordinaten kompensiert werden.

Anders als bei den Bases ist die Methode zum Vermessen des TCP recht aufwendig. Bei der am häufigsten verwendeten Bestimmungsart der Position, muss derselbe Punkt aus vier möglichst verschiedenen Richtungen mit dem Tool angefahren werden. Die Positionen des Flanschs bei den Messungen dürfen nicht auf einer Ebene liegen, da sonst mehrere Positionen für den TCP möglich sind. Da drei Punkte immer auf einer Ebene liegen, reichen hier drei Messungen für die drei Unbekannten, ΔX , ΔY und ΔZ , nicht aus.

Die Vermessung der Orientierung des Tools kann ähnlich wie die Bestimmung einer Basis ablaufen. Zunächst wird der TCP auf einen festen Punkt gefahren. Als Zweites wird derselbe Punkt mit einem Offset entgegen der Stoßrichtung des Tools angefahren, so wird die negative x-Achse bestimmt. Zuletzt wird der Punkt noch mit einer beliebigen positiven Verschiebung in der y-Achse angefahren. Die Berechnung in der Robotersteuerung verläuft dann genau wie bei der Vermessung einer Basis (Sh. 4.2.1).

Aufgrund der Aufwendigkeit dieses Verfahrens werden die Transformationen meist beim Entwurf des Greifers, im „Computer-aided Design“ (CAD), berechnet und beim Einrichten des Roboters in die Robotersteuerung eingetragen.

4.4 Lastdaten

Beim Anbauen von Aufsätzen ändert sich das Gewicht des Greifarms, und somit auch die Trägheit. Die Robotersteuerung verwendet die konfigurierten Lastdaten unter anderem zum Berechnen der Beschleunigung und somit indirekt auch bei der Geschwindigkeits- und Beschleunigungsüberwachung. „Deswegen ist es von größter Wichtigkeit, dass die Werkzeuglastdaten korrekt eingetragen werden.“ (KUKA Deutschland GmbH 2021, S. 117) Traglasten können über die Software „KUKA.LoadDataDetermination“ automatisch ermittelt werden, ansonsten müssen die Werte manuell, oder über CAD-Programme berechnet und von Hand eingetragen werden.

Da der Greifer während des Ablaufs durch Aufheben und Ablegen von Verpackungseinheiten ein variables Gewicht hat, müssen mehrere Sets an Lastdaten definiert werden, zwischen denen im KRL-Code nach Bedarf gewechselt wird.

4.5 Höhengsuchfahrt



Abb. 4.4: ECO-Packs auf Paletten
(www.blum.com/de/de/services/verpackungslogistik/verpackungsloesungen/)

Beim Beginn des Depalettierens einer neuen Palette ist der Roboteranlage die Anzahl der gestapelten Schichten nicht bekannt. In dieser Situation wird über GML eine sogenannte Höhengsuchfahrt angefordert. Bei einer Suchfahrt fährt der Roboter langsam und mit ausgefahrenem Höhenmesszylinder über der Palette nach unten. Sobald der Messzylinder die oberste Schicht der Palette berührt erkennt ein Sensor, dass der Zylinder komprimiert wird und der Roboter wird angehalten. Die Endhöhe des Roboters wird an die SPS übermittelt, die mithilfe der Formel (2), die

Anzahl der Teile auf der Palette berechnet und abspeichert.

Im Fall der Anlage FA3287 befinden sich auf der Palette vier ECO-Packs pro Schicht (Sh. Abb. 4.4). Um garantiert die oberste Schicht zu erkennen, wird bei der Höhengsuchfahrt der Mess-taster mittig über der Palette positioniert. Mit ausgefahrenem Messzylinder als ausgewähltem TCP beginnt die Suchfahrt also ähnlich wie in Formel (1) bei

$$\left(\frac{Palettenlaenge_1}{2} \middle| \frac{Palettenlaenge_2}{2} \middle| \Delta z \right) \quad (3)$$

Δz steht hier für die maximale Stapelhöhe inklusive eines Puffers. Das Ende der Suchfahrt befindet sich knapp über $z = 0$, um zu verhindern, dass als Stapelhöhe der Wert „0“ gemessen wird und eventuelle Fehler auftreten. Stattdessen wird der SPS übermittelt, dass die Suchfahrt nicht erfolgreich war.

5 Programmierung in GML

In GML wird der Hauptablauf des Roboters programmiert. Das Programm startet dabei immer mit einer Grundstellungsfahrt, sodass die Ausgangsposition bekannt ist. Durch Einbinden der Grundstellungsfahrt wird vom GML-Compiler auch der Aufruf an den Codebaustein zur Kommunikation mit der Robotersteuerung in den Hauptablauf eingefügt. Alternativ kann der Aufruf auch manuell eingefügt werden. Dadurch kann allerdings nicht erreicht werden, dass der Hauptablauf ohne Grundstellungsfahrt gestartet werden kann, da über die Bedienoberfläche FlexOP das Programm nur gestartet werden kann, wenn der Roboter sich bereits in der Grundstellung befindet. Dazu kann in FlexOP manuell eine Resetfahrt ausgeführt werden, unabhängig vom Hauptablauf.

5.1 Variablen

Die GML stellt Abläufe ähnlich dar, wie die BPMN. Gleich wie bei der Modelliersprache können direkt in GML keine Variablen deklariert und angefragt werden. Stattdessen können aber die im restlichen SPS-Projekt angelegten Variablen gelesen und geschrieben werden.

Dazu werden in einer globalen Variablenliste mitunter die folgenden Merker angelegt, die vom GML-Programm verwendet werden können. Die Variablen werden zum Teil auch vom Codebaustein des Greifers beschrieben, da dort die Signale der Sensoren verarbeitet werden und einige Merker, wie „PlacePosFree“, nur Abstraktionen der Sensormesswerte sind.

Variablenname	Datentyp	Verwendung
ChosenJob	INT	Zum Speichern des nächsten ausgewählten Jobs.
GripperHasItem	BOOL	Signal, dass sich im Greifer ein ECO-Pack befindet.
ItemAvailible	BOOL	Zeigt an, dass noch mindestens ein ECO-Pack auf der Palette liegt.
PlacePosFree	BOOL	„True“, wenn auf der GFT Platz für ein Item ist.

Abb. 5.1: Für GML eingerichtete globale Variablen

Abfragen von Variablen erfolgen in GML in Rautenförmigen Shapes. (Sh. Abb. 5.3 & 5.4) Die Abfragen müssen einzeln in der SPS-Programmiersprache ST programmiert werden. (Sh. Abb. 5.2) Innerhalb der Shapes, die Programmierung in ST ermöglichen sind komplexe Programme möglich. Um jedoch einen möglichst großen Teil der Logik über die grafische Darstellung der GML-Codes erkennen zu können soll der Code in ST möglichst kurz gehalten werden.

```
1 IF NOT gml.Reset THEN
2     IF gv_GmlState.ChosenJob = 1 THEN
3         gml.Jump := TRUE;
4     ELSE
5         gml.Ready := TRUE;
6     END_IF
7 END_IF
```

Abb. 5.2: Code der Abfrage „Job1?“, aus Abbildung 5.4

Über die Variablen „gml.Ready“ und „gml.Jump“ kann gesteuert werden, welcher der beiden Pfade aus dem Shape verwendet wird. Hierbei steht „gml.Ready“ für einen kontinuierlichen Durchlauf des Shapes, also wird der Ausgang gegenüber dem eingehenden Pfeil ausgewählt.

5.2 Auswahl der Jobs

Ausgehend von den booleschen Variablen aus der Tabelle 5.1 wird in dem GML-Unterprogramm ausgewählt, welcher Job als Nächstes auszuführen ist. Das Ergebnis wird in der Variable „ChosenJob“ gespeichert.

Wenn sich ein ECO-Pack im Greifer befindet, muss als nächster Job immer Nummer zwei ausgeführt werden, damit das Teil auf der GFT abgelegt wird. Also wird der Greiferinhalt als erstes angefragt, und die Jobnummer auf „2“ gesetzt, wenn der Greifer voll ist. (Sh. Abb. 5.3) Hier muss nicht auf eine freie GFT gewartet werden, da erst ein weiteres ECO-Pack aufgehoben wird, wenn die GFT wieder frei ist. Wenn der Greifer leer ist, muss entschieden werden, ob bereits ein neues ECO-Pack aufgehoben und wieder abgelegt werden kann. Das bedeutet, wenn auf der Palette mindestens ein Teil liegt und die Ablageposition wieder frei ist, wird über Job eins die Palette angefahren und das nächste ECO-Pack aufgehoben. Andernfalls wird die Warteposition angefahren. (Sh. Abb. 5.3)

Da die Jobauswahl mit dem anschließenden Ausführen zyklisch ausgeführt wird, wird beim Warten immer wieder die Jobauswahl ausgeführt. Solange die Palette leer ist, beziehungsweise die GFT belegt ist, wird also immer wieder der dritte Job ausgewählt und der Arm bleibt in der Warteposition, das er sich bereits am Ziel des Jobs befindet.

Bei der Jobauswahl in Abbildung 5.3 wird nicht auf die Position des Roboters geachtet, denn im Depalettierablauf wird der Arm nur über die drei Jobs vom Typ vier beziehungsweise die Grundstellungsfahrt bewegt. Innerhalb dieser Jobs sind die einzigen nicht unterstützten Wege von der Palette zur Warteposition und von der Warteposition zur GFT. Beide Szenarien können nicht auftreten. Der Job zur Warteposition wird nur mit leerem Greifer aufgerufen, was nicht der Fall ist, wenn im vorherigen Job ein ECO-Pack von der Palette aufgehoben wurde. Ebenso wird die GFT nur mit vollem Greiferaufsatz angefahren, was beim Anfahren der Warteposition über

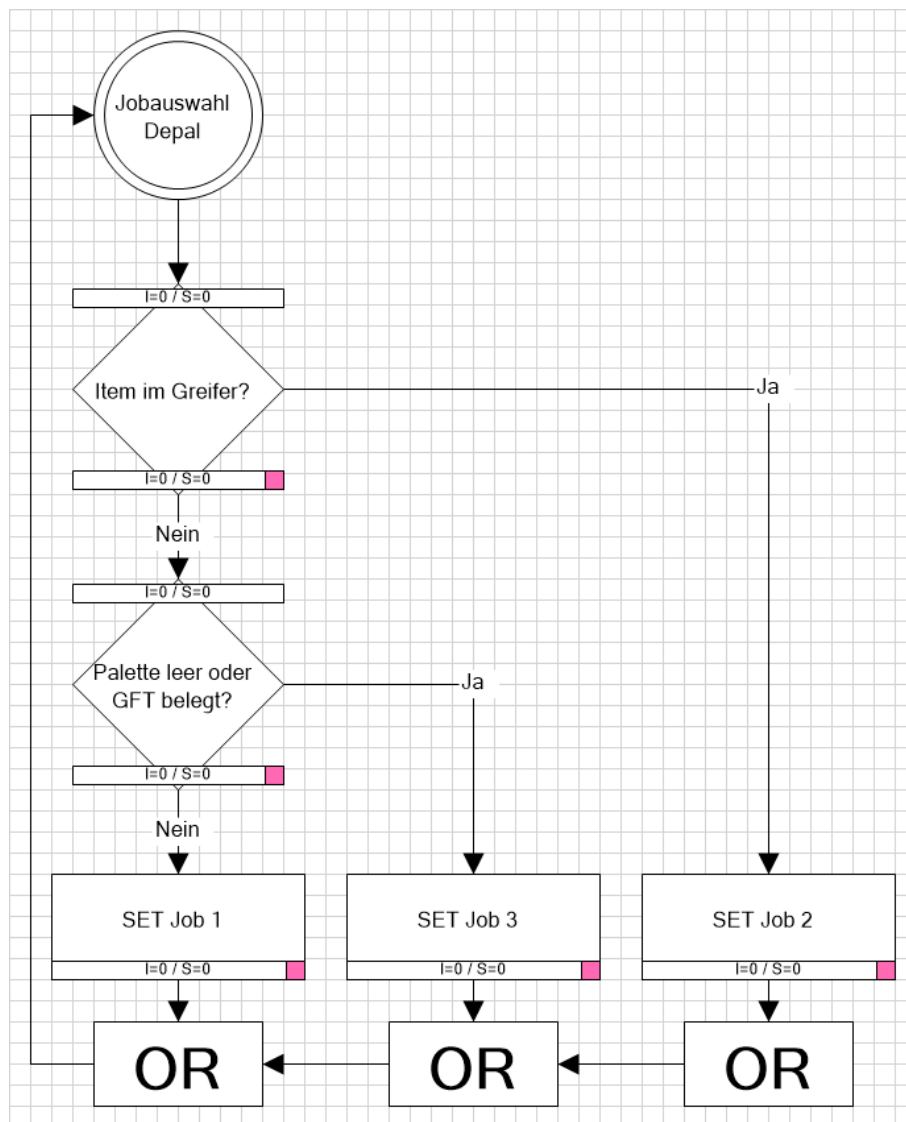


Abb. 5.3: GML-Code zur Auswahl eines Jobs

Job „3“ nicht möglich ist.

Die Grundstellung kann mithilfe der Resetfahrt aus allen Positionen angefahren werden, ebenso können alle Jobs des Typ „4“ aus der Grundstellung gestartet werden. So kann das Depalettierprogramm jederzeit abgebrochen und aus der Grundstellungsfahrt korrekt fortgesetzt werden.

5.3 Durchführung der Jobs

Der eigentliche Hauptablauf des Roboters beginnt, nachdem die Grundstellungsfahrt, am Beginn des GML-Programms durchgeführt wurde und zudem sichergestellt wurde, dass der Automatikbetrieb in FlexOP aktiviert wurde.

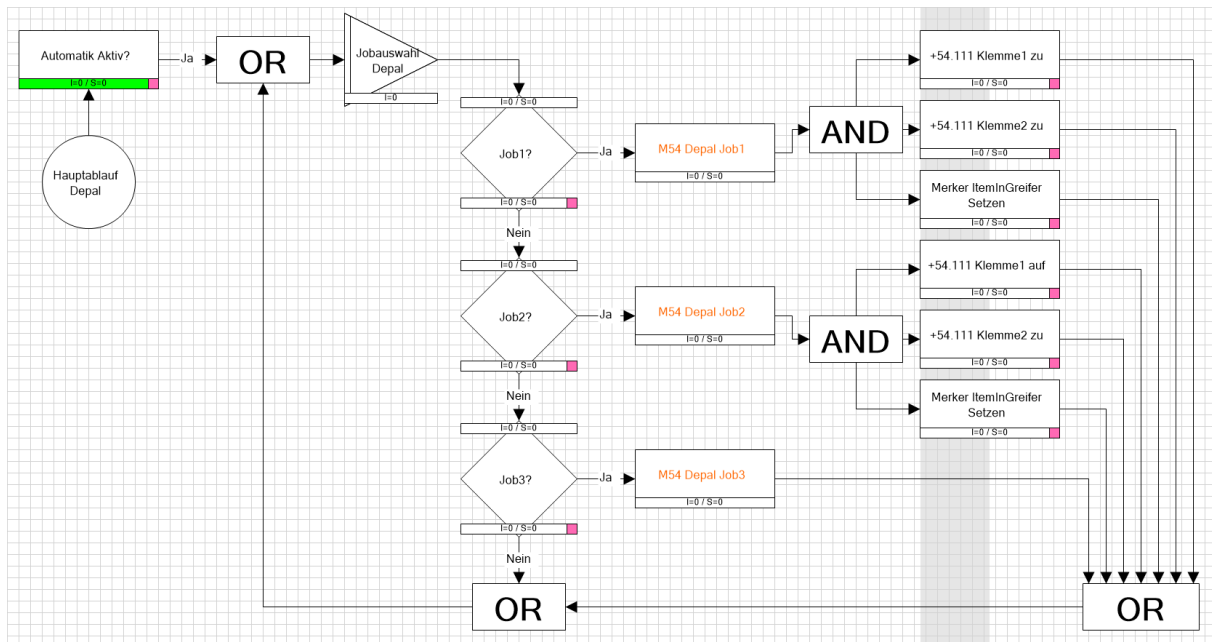


Abb. 5.4: Hauptablauf des Roboters in GML

Der Hauptablauf ruft zyklisch, das Unterprogramm zur Jobauswahl aus der Abbildung 5.3 auf. Anschließend wird über mehrere Abfragen der ausgewählte Job ermittelt und ausgeführt. In der Abbildung 5.4 vom Hauptablauf des Roboters, findet das Aufrufen der Jobs in den rechteckigen Shapes mit oranger Schrift statt. Nachdem die Jobs „1“ oder „2“ vollständig ausgeführt wurden, werden zudem noch die Klemmen des Greifers geöffnet, beziehungsweise geschlossen. Auch die Merkervariable, ob sich ein Teil in Greifer befindet wird beim Ablegen und Aufheben von ECO-Packs angepasst. Nach dem Ausführen vom dritten Job, zum Fahren in die Warteposition ist nichts Weiteres zu tun, also wird sofort mit der Berechnung des nächsten Jobs weiter gemacht.

6 Zusammenfassung zum Leitfaden

Abschließend zum Projekt, soll ein Leitfaden zur Erstellung von Depalettierabläufen angefertigt werden. Der Leitfaden soll zukünftigen Programmieren Hilfe bei der Umsetzung von Depalettierprogrammen bieten. Zudem soll durch Standardisierung des Vorgehens die Verständlichkeit der entstehenden Programme erhöht werden.

Der Leitfaden wird in der Sprache „Markdown“ (MD) erstellt. Ähnlich zu $\text{\LaTeX}$, kann MD zum Erstellen von PDF-Dateien verwendet werden. Verglichen mit $\text{\LaTeX}$, ist MD jedoch leichter zu schreiben und somit besser geeignet um Dokumente zu erstellen, die regelmäßig bearbeitet werden. Die einfache Bearbeitbarkeit ist wichtig, da der Leitfaden vermutlich nicht nur im Laufe des Projektes bearbeitet wird, sondern auch danach weiter angepasst wird, um weitere Informationen einzufügen, oder veraltete Informationen auszubessern.

Alternativ zu MD hätte auch „Microsoft Word“ (Word) zum Erstellen des Leitfadens verwendet werden können. Word bietet den Vorteil, dass nahezu jeder in der Lage ist die Software zu bedienen, wohingegen MD vergleichsweise selten verwendet wird. Dateien im „.docx“-Format sind jedoch nicht kompatibel mit Git, was die Versionierung erschwert. Markdown auf der anderen Seite wird nativ von Github unterstützt und kann als HTML gerendert angezeigt werden. Ein weiterer Vorteil von MD ist, dass Schriftart, Textgröße, Positionierung von Bildern und Tabellen und nahezu alle anderen Formaspekte automatisch festgelegt werden. Dadurch bleibt der Leitfaden einheitlich formatiert und somit übersichtlich. In Fällen, in denen das automatische Format nicht wünschenswert ist, kann das Aussehen auch manuell über HTML und CSS direkt in der Markdown-Datei angepasst werden.

6.1 Inhalt

Der Inhalt des Leitfadens ist ähnlich strukturiert, wie diese Arbeit. Zusätzlich werden Konzepte an konkreten Beispielen leichter verständlich gemacht.

Zu Beginn werden die nötigen Vorbereitungen dargestellt, also das Erstellen des Greiferbausteins und die Verknüpfung mit den Ein- und Ausgängen. Hierbei wird genauer darauf eingegangen, wie aus dem EPLAN zu erkennen ist, welche Aktoren und Sensoren am Greifer vorhanden sind und wie diese im Code möglichst leicht umgesetzt werden können. Wie die Variablen im Sys-Manager zu verknüpfen sind, wird anhand von Beispielen verständlich gemacht. Zudem wird erklärt, wie der sogenannte E/A-Check durchgeführt wird. Im E/A-Check wird kontrolliert, ob die Verknüpfung der Variablen korrekt funktioniert hat. Zuletzt werden im ersten Abschnitt noch einige Tipps eingefügt, wie verschiedene Ursachen für Fehler beim E/A-Check gefunden und behoben werden können.

Im zweiten Teil des Leitfadens wird die Programmierung in KRL erklärt. Dazu gehören auch die Erstellung von Basen, TCPs Lastdaten und ähnlichen Konfigurationen, die nötig sind um den

Roboter optimal zu programmieren. In diesem Abschnitt liegt ein besonderer Fokus auf Verallgemeinerung, da für jeden Depalettierablauf andere Umstände gegeben sind. Beispielsweise wird im Projekt „FA3278“ von einer Palette auf eine GFT depalettiert. In anderen Projekten sind eventuell mehrere Paletten oder Ablagepunkte gegeben. Aufgrund der anderen Anforderungen kann kein einzelnes Set von Jobs für alle Projekte verwendet werden. Also wird in dem Kapitel erklärt, wie aus den nötigen Positionen sinnvolle Jobs definiert werden können. Zudem werden hier Faustregeln angegeben, wie beispielsweise „für jede Palette eine eigene Basis erstellen“.

Im dritten Abschnitt wird die Programmierung in GML dargestellt. Auch hier kann keine allgemeine Lösung für alle Depalettierprojekte angeboten werden. Stattdessen wird zunächst die Funktionsweise von GML erklärt, da einige Anlagenprogrammierer bei fpt noch nie mit einer vergleichbaren Sprache gearbeitet haben. Anschließend wird das Vorgehen, dass von Blum verwendet wird, vorgestellt. Zuerst wird gezeigt, wie mithilfe der Sensoreingänge und eventuell Zustandsvariablen ein verfügbarer Job ausgewählt wird. Danach wird das Shape zum Ausführen von Jobs vorgestellt, hier wird besonders auf die verschiedenen Konfigurationsmöglichkeiten und deren Verwendung eingegangen. Zuletzt wird kurz das Ausführen von Jobs, zusammen mit dem Betätigen der dazugehörigen Aktoren, erklärt

6.2 Anhänge

Im Leitfaden werden einige Code-Beispiele gezeigt. Zusätzlich dazu werden einige Dateien als Anhang hinzugefügt. Die Codedateien können als ausführlichere Beispiele bei der Programmierung helfen. Zusätzlich kann das Programm bei ähnlichen Projekten mit einiger Anpassung wieder verwendet werden. Dazu werden nach Möglichkeit alle projektspezifischen Elemente aus den Erstellten Codedateien entfernt.

Der Anhang kann zudem nach weiteren Depalettierprojekten erweitert werden. So entsteht eine stetig wachsende Bibliothek von konfigurierbaren Grundgerüsten für verschiedene Archetypen von Depalettierabläufen. Aus der Sammlung können zukünftig passende Entwürfe ausgewählt und für neue Projekte angepasst werden. Wenn beim Beginn eines neuen Projekts noch kein passender Ablauf in der Bibliothek zu finden ist, muss das Depalettierprogramm zwar von Grund auf neu erstellt werden, nach der Fertigstellung kann der Entwurf dann im Anhang des Leitfadens archiviert werden. So müssen ähnliche Anlagen nicht mehr von Vorne programmiert werden, sondern können auf dem Code der alten Projekte aufgebaut werden, was voraussichtlich Zeit spart. Durch die Verwendung einheitlicher Grundlagen für ähnliche Depalettierabläufe wird auch die Einheitlichkeit des Programmierstils erhöht.

6.3 Verteilung

Um den Leitfaden allen Anlagenprogrammierern der fpt zugänglich zu machen wird das Markdown-Dokument inklusive des Anhangs in ein eigenes Git-Repository hochgeladen. Das Repository wird wie alle anderen Repositories bei fpt über Github gehostet. So kann das Anzeigen der Leitfadenversion in MD von Github im Browser übernommen werden. Zum Lesen des Leitfadens lokal auf dem Rechner wird auch die eine, ins PDF-Format, kompilierte Version des Leitfadens hochgeladen.

Um das Aktualisieren der kompilierten Datei zu automatisieren wird eine GitHub-Action erstellt. So wird jede Änderung an der Markdown-Version automatisch in die PDF-Datei übernommen. Zudem kann bei jedem Commit geprüft werden, dass keine Kompilierfehler auftreten, andernfalls kann der Commit blockiert werden. Das garantiert, dass der Leitfaden immer intakt bleibt.

Unter der Annahme, dass alle Anlagenprogrammierer bei fpt vernünftig mit dem Repository umgehen, kann dem Team der Anlagenprogrammierer die Berechtigungen zum Bearbeiten des Repositories gegeben werden. Das Feature zum Erstellen von Teams und die Vergabe von Berechtigungen an ganze Teams wird schon bei fpt genutzt und das Anlagenprogrammierer-Team ist bereits eingerichtet.

7 Reflexion

Nach der Darstellung der Projektarbeit wird im Folgenden der erreichte Stand zusammengefasst und kritisch beurteilt. Es wird hier insbesondere darauf eingegangen, welche Anforderungen nicht erfüllt wurden und welche Ergänzungen nötig sind, sodass der Depalettierablauf des Moduls 54 in der Anlage FA3278 funktionsfähig ist.

7.1 Zusammenfassung

Der Codebaustein für den Greifer ist vollständig in ST implementiert und die Variablen würden über den Systemmanager mit den Sensoren und Aktoren der Roboteranlage verknüpft. Die nötigen Referenzen zum SPS-Code wurden in der Projektdatenbank eingetragen. Damit kann der Roboterarm über FlexOP bereits von Hand bewegt werden. Über FlexOP wurde anschließend getestet, dass die Variablen richtig verknüpft wurden, beziehungsweise die Kabel an der Roboteranlage richtig verbunden sind.

In KRL wurden alle drei Jobs im Typ vier implementiert, mit Ausnahme der Strecken die in der Home-Position starten. Die Strecken wurden nicht implementiert, da beim Beenden der Arbeit am KRL-Programm noch nicht bekannt war, wo die Grundstellung sein soll. Die Teachpositionen und auch die Basis der Palette sind bereits angelegt, haben aber noch nicht die korrekten Werte, da das Plugin, das Blum zum Teachern verwendet, nicht funktioniert hat. Aus diesem Grund konnten die Jobs noch nicht auf Kollisionsfreiheit getestet werden.

Die Höhengsuchfahrt wurde nahezu komplett aus dem Projekt „FA3177“ übernommen, und sollte daher korrekt implementiert sein. Das sogenannte „Depalettierschema“ wurde aus Zeitgründen in dieser Arbeit nicht beachtet und fehlt somit ganzheitlich.

Der Hauptablauf in GML ist vollständig implementiert, wie er in den Abbildungen 5.3 und 5.4 zu sehen ist. Dazu wurden auch die nötigen Variablen aus Abbildung 5.1 in der globalen Variablenliste angelegt.

7.2 Nötige Ergänzungen

Diese Projektarbeit musste, aufgrund von unglücklicher Zeitplanung in nur sechs Wochen abgeschlossen werden. Die Aufgabenstellung war allerdings für einen Bearbeitungszeitraum von 12 Wochen ausgelegt. Am Ende der Arbeit sollte der Depalettierablauf vollständig implementiert und getestet sein. Aufgrund des Zeitmangels konnten nicht alle nötigen Schritte durchgeführt werden. Die noch fehlenden Implementationen werden voraussichtlich nach Abgabe der Arbeit, parallel zur Vorbereitung der Präsentation vorgenommen.

Nachträglich zum Projekt müssen zunächst die Strecken zum Ausfahren aus der Grundstellung in die Jobs des Depalettier-Typs implementiert werden. Das Vorgehen hierbei ist identisch

zum Programmieren der anderen Strecken, die bereits fertig sind. Um die Strecken testen zu können müssen die nötigen Positionen, sowie die Basis der Palette geteacht werden. Die Positionen können durch einfaches Anfahren im manuellen Betrieb festgelegt werden. Das Vorgehen zum Anlegen der Basis ist in Kapitel 4.2.1 detailliert beschrieben.

Im letzten Schritt muss noch das Depalettierschema in den Ablauf integriert werden. Das Depalettierschema bestimmt die Reihenfolge in der Teile von der Palette genommen werden und unter anderem auch die Orientierung, die der Greifer dafür benötigt. Die Informationen werden in FlexOP aus einer XML-Datei ausgelesen und durch die SPS an das Programm in KRL übergeben. Offsets, zum Anfahren der Position des nächsten Teils, werden im Programm der Robotersteuerung bereits beachtet, wohingegen die Parameter „Orientation“ und „Approach“ noch nicht im KRL-Programm verwendet werden.

8 Fazit

Ausgehend von der ursprünglichen Aufgabenstellung kann das Projekt höchstens als teilweiser Erfolg bewertet werden, da einige Requirements für eine funktionstüchtige Anlage nicht erfüllt wurden. Unter Berücksichtigung der großen Diskrepanz zwischen der eingeplanten Zeit und der verfügbaren Zeit kann die Arbeit jedoch als Erfolg eingestuft werden.

Über den nächsten Monat wird höchstwahrscheinlich weiter an der Anlage gearbeitet werden, sodass möglichst bald das fertige Produkt an Blum übergeben werden kann. Voraussichtlich kann der Depalettierablauf noch vor Beginn der Theoriephase fertiggestellt werden. Andernfalls werden die übrigen Arbeiten vermutlich über den Oktober von einem anderen Anlagenprogrammierer übernommen werden.

Literatur

FPT Robotik GmbH Co. KG (2023). *FA3278_Blum_IL43G Palettieren - EPLAN*.

STIWA Holding GmbH (2024). *Graphic Motion Language*. URL: <https://www.stiwa.com/software/engineering/graphic-motion-language>.

FPT Robotik GmbH Co. KG (2024). *FA3278_Blum_IL43G Palettieren - GreiferM54_Lastdaten*.

KUKA Deutschland GmbH (2021). *Roboterprogrammierung 1 - Schulungsunterlagen*.

A Anhang

Anhang.zip

└─ Quellen

└─ FA3278_Blum_IL43G_Palettieren_EPLAN.pdf

└─ FA3278_Blum_IL43G_Palettieren_GreiferM54_Lastdaten_V1.pdf

└─ Graphic_Motion_Language.lnk

└─ Graphic_Motion_Language.htm

Aus Geheimhaltungsgründen dürfen die Schulungsdokumente der KUKA Deutschland GmbH nicht als Quellennachweis angefügt werden.